



GSM8k Solution Verifier

ProblemFormailzer

▼ Prompt

Hãy viết giúp tôi file 'Formalizer/ProblemFomalizer.py'. File này có nhiệm vụ nhận đề bài trong file 'Input/Problem.txt' cho qua model LLM với OpenRouter để tạo các thực thể có giá trị là số đã được nói trong đề bài. Nó không thực hiện tính toán các thực thể được nhắc tới nhưng không có số và phải tính toán. Tôi muốn mỗi thực thể có 4 trường:

- 'value' là giá trị (bắt buộc là số) trong đề bài cho (nếu đề bài cho là chữ thì hãy chuyển thành số).
- 'unit' là đơn vị của thực thể (dollar, days, ...) nhưng 'unit' có thể là rỗng với các thực thể mang tính scalar trong đề bài (ví dụ: phân số, phần trăm, nhân tử, ...)
- 'location' là vị trí của thực thể (ở đầu bài thì là 'input', được tính toán ra ở bước i thì là 'stepi' (stepi khác step cuối), đáp số cần tìm thì là 'target').
- 'grand_unit' là đơn vị của thực thể được đối chiếu theo target (ví dụ 'unit' của sách là books, 'unit' của vở là pens nhưng 'unit' là tổng sách và bút (target) lại là items thì 'grand_unit' của sách và bút sẽ là items nhưng nếu target là tổng số bút ('unit' là pens) thì 'grand_unit' của sách vẫn là 'books' vì không liên quan nên sách không liên quan). Các thực thể

mang tính scalar trong đề bài (ví dụ: phân số, phần trăm, nhân tử, ...) cũng có thể để 'grand_unit' là rỗng.

Phần này chỉ formalize đề bài và không thực hiện tính toán nên chỉ có các thực thể trong đề và 'location' của các thực thể này chỉ là 'input' và 'target', riêng thực thể có 'location' là 'target' thì sẽ có 'value' là rỗng. Tôi muốn file này cho output vào file 'Output/ProblemEntities.yaml'.

▼ One-shot

▼ Input

Ví dụ bài toán trong file 'Input/Problem.txt':

"Nancy buys 2 coffees a day. She grabs a double espresso for \$3.00 every morning. In the afternoon, she grabs an iced coffee for \$2.50. After 20 days, how much money has she spent on coffee?"

▼ Output

Tôi muốn output trong file 'Output/ProblemEntities.yaml' là danh sách các thực thể như sau:

morning_coffee_price:

'value': 3.00

'unit': dollars

'location': input

'grand_unit': dollars

afternoon_coffee_price:

'value': 2.50

'unit': dollars

'location': input

'grand_unit': dollars

days:

'value': 20

'unit': days

'location': input

'grand_unit':

total_cost:

'value':

'unit': dollars

'location': target

'grand_unit': dollars

Nếu formalize thành công, chương trình sẽ ghi "Pass ProblemFormalizer", ngược lại nếu thất bại sẽ ghi "Fail ProblemFormalizer" vào 'Output/Log.yaml'. Sau khi tạo xong, 'Formalizer/ProblemFormalizer.py' sẽ copy file 'Output/ProblemEntities.yaml' vào file 'Output/PlanEntities.yaml' và 'Output/StudentAnswerEntities.yaml' bằng code python.

▼ Code

```
import os
import re
import shutil
from pathlib import Path

import requests
import yaml
from dotenv import load_dotenv

load_dotenv()

INPUT_PATH = Path("Input/Problem.txt")
OUTPUT_PATH = Path("Output/ProblemEntities.yaml")
PLAN_PATH = Path("Output/PlanEntities.yaml")

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
MODEL = "openai/gpt-4o-mini"

IMPORTANT_RULES = """
Quy tắc bắt buộc:
- Chỉ trích xuất các số, phân số, phân trăm, hệ số được nêu TRỰC TIẾP trong đề bài.
- Tuyệt đối không tính toán.
- Tuyệt đối không tạo biến trung gian.
- Không tạo các biến kiểu: after_first, after_second, ta_bs_after_first_closing, tabs_closed_first...
- Nếu một giá trị phải suy ra bằng phép tính thì KHÔNG được đưa vào output.
- Các phân số/phân trăm/hệ số trong đề vẫn được giữ lại
```

dưới dạng scalar input.

- Nếu đề bài có "1/4", hãy chuyển value thành 0.25.
 - Nếu đề bài có "2/5", hãy chuyển value thành 0.4.
 - Nếu đề bài có "half", hãy chuyển value thành 0.5.
 - Nếu đề bài có "twice", hãy chuyển value thành 2.
 - Nếu đề bài có "three times", hãy chuyển value thành 3.
 - Thực thể target là đại lượng cần tìm, value luôn là chuỗi rỗng "".
 - location chỉ được là "input" hoặc "target".
- """

```
def read_problem() -> str:
    if not INPUT_PATH.exists():
        raise FileNotFoundError(f"Không tìm thấy file:
{INPUT_PATH}")

    return INPUT_PATH.read_text(encoding="utf-8").strip
()
```

```
def build_prompt(problem: str) -> str:
    return f"""
Bạn là một Problem Formalizer.
```

Nhiệm vụ:

Từ đề bài toán, hãy tạo danh sách các thực thể xuất hiện trong đề.

Mỗi thực thể có đúng 3 trường:

- value: giá trị số. Nếu đề bài viết bằng chữ thì chuyển thành số.
- unit: đơn vị của thực thể, ví dụ dollars, days, tabs, apples. Nếu là scalar như phân số, phần trăm, hệ số nhân thì để rỗng "".
- location:
 - "input" nếu là dữ kiện đã cho trực tiếp trong đề.
 - "target" nếu là đáp số cần tìm.

`{IMPORTANT_RULES}`

Ví dụ 1:

Đề bài:

Nancy buys 2 coffees a day. She grabs a double espresso for \$3.00 every morning. In the afternoon, she grabs an iced coffee for \$2.50. After 20 days, how much money has she spent on coffee?

Output:

`morning_coffee_price:`

`value: 3.00`

`unit: dollars`

`location: input`

`afternoon_coffee_price:`

`value: 2.50`

`unit: dollars`

`location: input`

`days:`

`value: 20`

`unit: days`

`location: input`

`total_cost:`

`value: ""`

`unit: dollars`

`location: target`

Ví dụ 2:

Đề bài:

Jose had 400 tabs opened up in his windows browser. After about one hour of browsing, he closed 1/4 of the tabs to allow easy browsing. He read some news from differen

t pages, then closed $\frac{2}{5}$ of the remaining tabs. If he closed half of the remaining tabs after further analyzing some pages, how many windows tabs did he have remaining open?

Output:

initial_tabs:

value: 400

unit: tabs

location: input

first_closed_fraction:

value: 0.25

unit: ""

location: input

second_closed_fraction:

value: 0.4

unit: ""

location: input

third_closed_fraction:

value: 0.5

unit: ""

location: input

remaining_tabs:

value: ""

unit: tabs

location: target

Bây giờ hãy formalize đề bài sau.

Chỉ output YAML hợp lệ.

Không dùng markdown.

Không giải thích thêm.

Đề bài:

```

{problem}
"".strip()

def call_openrouter(prompt: str) -> str:
    if not OPENROUTER_API_KEY:
        raise EnvironmentError(
            "Không tìm thấy OPENROUTER_API_KEY. Hãy kiểm tra file .env."
        )

    response = requests.post(
        url="https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {OPENROUTER_API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": MODEL,
            "messages": [
                {
                    "role": "system",
                    "content": (
                        "You are a strict YAML generator. "
                        "You extract only directly stated entities from math word problems. "
                        "Never solve the problem."
                    )
                },
                {
                    "role": "user",
                    "content": prompt,
                },
            ],
            "temperature": 0,
        }
    )

```

```

        },
        timeout=60,
    )

    response.raise_for_status()
    data = response.json()

    return data["choices"][0]["message"]["content"]

def clean_yaml_text(text: str) -> str:
    text = text.strip()

    text = re.sub(r"^``yaml\s*", "", text)
    text = re.sub(r"^``\s*", "", text)
    text = re.sub(r"\s*``$", "", text)

    return text.strip()

def validate_entities(entities: dict) -> None:
    if not isinstance(entities, dict):
        raise ValueError("Output YAML phải là dictionary.")

    for entity_name, fields in entities.items():
        if not isinstance(fields, dict):
            raise ValueError(f"Thực thể '{entity_name}' không hợp lệ.")

        required_fields = {"value", "unit", "location"}

        if set(fields.keys()) != required_fields:
            raise ValueError(
                f"Thực thể '{entity_name}' phải có đúng 3 trường: value, unit, location."
            )

```



```

        if fields["location"] not in {"input", "target"}:
            raise ValueError(
                f"location của {entity_name} chỉ được là 'input' hoặc 'target'."
            )

        if fields["location"] == "target":
            if fields["value"] not in ("", None):
                raise ValueError(
                    f"Thực thể target {entity_name} phải có value rỗng."
                )

            if fields["location"] == "input":
                if fields["value"] in ("", None):
                    raise ValueError(
                        f"Thực thể input {entity_name} phải có value là số."
                    )

                if not isinstance(fields["value"], (int, float)):
                    raise ValueError(
                        f"value của {entity_name} phải là số, hiện tại là {fields['value']}."
                    )

def save_yaml(entities: dict) -> None:
    OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)

    with OUTPUT_PATH.open("w", encoding="utf-8") as file:
        yaml.safe_dump(
            entities,
            file,

```

```

        allow_unicode=True,
        sort_keys=False,
        default_flow_style=False,
    )

def copy_to_plan_entities() -> None:
    shutil.copyfile(OUTPUT_PATH, PLAN_PATH)

def main() -> None:
    problem = read_problem()
    prompt = build_prompt(problem)

    raw_output = call_openrouter(prompt)
    yaml_text = clean_yaml_text(raw_output)

    entities = yaml.safe_load(yaml_text)

    validate_entities(entities)
    save_yaml(entities)
    copy_to_plan_entities()

    print(f"Saved: {OUTPUT_PATH}")
    print(f"Copied: {PLAN_PATH}")

if __name__ == "__main__":
    main()

```

Planner

▼ Prompt

Hãy viết giúp tôi file 'Formalizer/Solver/Planner.py'. File này có nhiệm vụ nhận đề bài trong file 'Input/Problem.txt' và nhận các thực thể của đề bài trong file 'Output/ProblemEntities.yaml' cho qua model LLM với OpenRouter

để sinh ra kế hoạch thực thi bài toán. Tôi muốn kế hoạch được đặt tên các bước theo thứ tự, mỗi bước gồm trường

- 'expr' là biểu thức thể hiện quan hệ tính toán giữa các thực thể.
- 'result' tên thực thể được tạo ra khi thực hiện phép tính đó
- 'result_unit' là đơn vị của thực thể đó. Các thực thể mang tính scalar trong đề bài (ví dụ: phân số, phần trăm, nhân tử, ...) cũng có thể để 'result_unit' là rỗng.
- 'result_grand_unit' là đơn vị của thực thể được đối chiếu theo target (ví dụ 'unit' của sách là book, 'unit' của vở là pen nhưng 'unit' là tổng sách và bút (target) lại là items thì grand_unit' sẽ là items). Các thực thể mang tính scalar trong đề bài (ví dụ: phân số, phần trăm, nhân tử, ...) cũng có thể để 'result_grand_unit' là rỗng.

Tôi muốn file này cho output vào file 'Output/Plan.yaml'. Khi tạo xong 'Output/Plan.yaml', hãy thêm các thực thể 'result' vào file 'Output/PlanEntities.yaml' (thêm tất cả trường 'unit' theo 'result_unit', 'location' là step thực thể đó được tính toán (chỉ thêm vào các thực thể chưa có 'location'), 'grand_unit' theo 'result_grand_unit') Ngoài ra, với các bước đặc biệt như bước đổi đơn vị thì hãy thêm cả trường 'value' vào biến mới được tạo ra khi đổi đơn vị. Tất cả việc thêm đều tạo bằng code python. Nếu sinh plan thành công, chương trình sẽ ghi "Pass Planner", ngược lại nếu thất bại sẽ ghi "Fail Planner" vào 'Output/Log.yaml'.

▼ One-shot

▼ Input

Ví dụ với bài toán trong file 'Input/Problem.txt':

"Nancy buys 2 coffees a day. She grabs a double espresso for \$3.00 every morning. In the afternoon, she grabs an iced coffee for \$2.50. After 20 days, how much money has she spent on coffee?"

Danh sách thực thể trong file 'Output/ProblemEntities.yaml':

morning_coffee_price:

'value': 3.00

'unit': dollars

'location': input

'grand_unit': dollars

afternoon_coffee_price:

'value': 2.50

'unit': dollars

'location': input

'grand_unit': dollars

days:

'value': 20

'unit': days

'location': input

'grand_unit':

total_cost:

'value':

'unit': dollars

'location': target

'grand_unit': dollars

▼ Output

Tôi muốn output trong file 'Output/Plan.yaml' là danh sách các bước như sau:

step1:

'expr': morning_coffee_price + afternoon_coffee_price

'result': daily_cost

'result_unit': dollars

'grand_result_unit': dollars

step2:

'expr': daily_cost * days

'result': total_cost

'result_unit': dollars

'grand_result_unit': dollars

▼ Code

```
import os
import re
import json
from pathlib import Path
from typing import Dict, Any

import yaml
```

```

import requests

from dotenv import load_dotenv

load_dotenv()

ROOT = Path(__file__).resolve().parents[2]

PROBLEM_PATH = ROOT / "Input" / "Problem.txt"
PLAN_PATH = ROOT / "Output" / "Plan.yaml"

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_MODEL = os.getenv("OPENROUTER_MODEL", "openai/gpt-4o-mini")

class Planner:
    def __init__(self):
        if not OPENROUTER_API_KEY:
            raise EnvironmentError("Missing OPENROUTER_API_KEY")

    def read_problem(self) -> str:
        return PROBLEM_PATH.read_text(encoding="utf-8").strip()

    def build_prompt(self, problem: str) -> str:
        return f"""
You are a math word problem planner.

Given a word problem, create a calculation plan.

Problem:
{problem}

Return ONLY valid JSON.

```

Format:

```
{{
  "step1": {{
    "expr": "entity_a + entity_b",
    "result": "new_entity",
    "result_unit": "unit"
  }},
  "step2": {{
    "expr": "new_entity * entity_c",
    "result": "target_entity",
    "result_unit": "unit"
  }}
}}
```

Rules:

- Step names must be step1, step2, step3, ...
- expr must use clear variable names.
- result is the variable created by that step.
- result_unit is the unit of result.
- Do not calculate numeric values.
- The final step result must answer the question.
- Use meaningful variable names based on the problem.
- For unit conversion, write expressions like:
minutes / 60
hours * 60
dollars * 100

Example:

Problem:

Nancy buys 2 coffees a day. She grabs a double espresso for \$3.00 every morning.

In the afternoon, she grabs an iced coffee for \$2.50.

After 20 days, how much money has she spent on coffee?

Output:

```
{{
```

```

    "step1": {{
        "expr": "morning_coffee_price + afternoon_coffee_price",
        "result": "daily_cost",
        "result_unit": "dollars"
    }},
    "step2": {{
        "expr": "daily_cost * days",
        "result": "total_cost",
        "result_unit": "dollars"
    }}
}}
"".strip()

def call_llm(self, prompt: str) -> str:
    response = requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {OPENROUTER_API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": OPENROUTER_MODEL,
            "messages": [
                {
                    "role": "system",
                    "content": "You output strict JSON only."
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            "temperature": 0
        },

```

```

        timeout=60
    )

    if response.status_code != 200:
        raise RuntimeError(
            f"OpenRouter error {response.status_code}: {response.text}"
        )

    return response.json()["choices"][0]["message"]
["content"]

def parse_json_output(self, raw_output: str) -> Dict[str, Any]:
    text = raw_output.strip()

    if text.startswith("`json`"):
        text = text[len("`json`"):].strip()

    if text.startswith("`"):
        text = text[len("`"):].strip()

    if text.endswith("`"):
        text = text[:-3].strip()

    try:
        data = json.loads(text)
    except json.JSONDecodeError as e:
        raise ValueError(f"LLM did not return valid JSON:\n{text}") from e

    if not isinstance(data, dict):
        raise ValueError("Plan must be a JSON object")

    return data

def validate_plan(self, plan: Dict[str, Any]) -> Non

```



```

e:
    index = 1

    for step_name, step in plan.items():
        expected_name = f"step{index}"

        if step_name != expected_name:
            raise ValueError(
                f"Invalid step name. Expected {expected_name}, got {step_name}"
            )

        if not isinstance(step, dict):
            raise ValueError(f"{step_name} must be a dictionary")

        for field in ["expr", "result", "result_unit"]:
            if field not in step:
                raise ValueError(f"{step_name} is missing field: {field}")

            if not isinstance(step[field], str):
                raise ValueError(f"{step_name}.{field} must be a string")

        index += 1

    def write_yaml(self, path: Path, data: Dict[str, Any]) -> None:
        path.parent.mkdir(parents=True, exist_ok=True)

        with open(path, "w", encoding="utf-8") as f:
            yaml.safe_dump(
                data,
                f,
                allow_unicode=True,
                sort_keys=False,

```

```

        default_flow_style=False
    )

    def run(self) -> None:
        problem = self.read_problem()

        prompt = self.build_prompt(problem)
        raw_output = self.call_llm(prompt)

        plan = self.parse_json_output(raw_output)
        self.validate_plan(plan)

        self.write_yaml(PLAN_PATH, plan)

        print(f"Saved plan to {PLAN_PATH}")

if __name__ == "__main__":
    planner = Planner()
    planner.run()

```

Executor

▼ Prompt

Hãy viết giúp tôi file 'Formalizer/Solver/Executor.py'. File này có nhiệm vụ nhận các thực thể trong file 'Output/PlanEntities.yaml' và các bước kế hoạch trong file 'Output/Plan.yaml'. Nó sẽ thực thi theo kế hoạch trong file 'Output/Plan.yaml' bằng code python. Nó sẽ gắn thêm trường 'reported_expr' là các value được tính theo bước vào các bước trong 'Output/Plan.yaml'. Sau khi gắn xong, nó sẽ gắn 'value' theo giá trị sau dấu "=" của 'reported_expr' của các thực thể chưa có value trong file 'Output/PlanEntities.yaml'. Ngoài ra nó còn thêm trường 'expr' cho các thực thể trong file Output/PlanEntities.yaml. Nếu thực thể có 'location' là input nó sẽ để trống 'expr', với các thực thể là 'result' trong các bước tính của file 'Output/Plan.yaml' 'expr' là trường 'expr' trong file 'Output/Plan.yaml'. Sau đó nó sẽ thêm trường 'formalized_expr' là trường chuẩn hoá 'expr' về phép tính

của các thực thể chỉ có 'location' là input, nếu thực thể có 'location' là input thì nó sẽ để trống 'formalized_expr' .

▼ One-shot

▼ Input

Ví dụ danh sách thực thể trong file 'Output/PlanEntities.yaml':

morning_coffee_price:

'value': 3.00

'unit': dollars

'location': input

'grand_unit': dollars

afternoon_coffee_price:

'value': 2.50

'unit': dollars

'location': input

'grand_unit': dollars

days:

'value': 20

'unit': days

'location': input

'grand_unit':

total_cost:

'value':

'unit': dollars

'location': target

'grand_unit': dollars

daily_cost:

'value':

'unit': dollars

'location': step1

'grand_unit': dollars

Kế hoạch trong file 'Output/Plan.yaml':

step1:

'expr': morning_coffee_price + afternoon_coffee_price

'result': daily_cost

```
'result_unit': dollars
'grand_result_unit': dollars
```

step2:

```
'expr': daily_cost * days
'result': total_cost
'result_unit': dollars
'grand_result_unit': dollars
```

▼ Output

Tôi muốn kế hoạch trong file 'Output/Plan.yaml' sẽ được update thành:

step1:

```
'expr': morning_coffee_price + afternoon_coffee_price
'result': daily_cost
'result_unit': dollars
'grand_result_unit': dollars
'reported_expr': 3.00 + 2.50 = 5.50
```

step2:

```
'expr': daily_cost * days
'result': total_cost
'result_unit': dollars
'grand_result_unit': dollars
'reported_expr': 5.50 * 20 = 110
```

Sau đó chương trình sẽ update file 'Output/PlanEntities.yaml' thành:

morning_coffee_price:

```
'value': 3.00
'unit': dollars
'location': input
'grand_unit': dollars
'expr':
'formalized_expr':
```

afternoon_coffee_price:

```
'value': 2.50
'unit': dollars
'location': input
'grand_unit': dollars
'expr':
'formalized_expr':
```

```

days:
  'value': 20
  'unit': days
  'location': input
  'grand_unit':
  'expr':
  'formalized_expr':

total_cost:
  'value': 110
  'unit': dollars
  'location': target
  'grand_unit': dollars
  'expr': daily_cost * days
  'formalized_expr': (morning_coffee_price + afternoon_coffee_price)
* days

daily_cost:
  'value': 5.50
  'unit': dollars
  'location': step1
  'grand_unit': dollars
  'expr': morning_coffee_price + afternoon_coffee_price
  'formalized_expr': morning_coffee_price + afternoon_coffee_price

```

Sau khi tạo xong các file, nó sẽ gọi file 'Verify/InsideChecker.py'. Nếu 'Verify/InsideChecker.py' in lỗi vào 'Output/Error.yaml' thì nó phải gọi lại LLM với input là các file 'Input/Problem.txt', 'Output/PlanEntities.yaml', 'Output/Plan.yaml', 'Output/Error.yaml', output là các file này đã được sửa (nếu file nào bị sai). Nó sẽ lặp lại đến khi 'Verify/InsideChecker.py' không in lỗi vào 'Output/Error.yaml'

▼ Code

StudentAnswerFormalizer

▼ Prompt

Hãy viết giúp tôi file 'Formalizer/StudentAnswerFormalizer.py'. File này có nhiệm vụ nhận đề bài trong file 'Input/Problem.txt', danh sách thực thể đề bài trong file 'Output/ProblemEntities.yaml' và bài làm học sinh trong file 'Input/StudentAnswer.txt' cho qua model LLM với OpenRouter để tạo ra file

'Output/StudentPlan.yaml' là kế hoạch tính toán của học sinh, kế hoạch này được đặt tên các bước theo thứ tự, mỗi bước gồm trường

- 'expr' là biểu thức thể hiện quan hệ tính toán giữa các thực thể.
- 'result' tên thực thể được tạo ra khi thực hiện phép tính đó.
- 'result_unit' là đơn vị của thực thể đó. Các thực thể mang tính scalar trong đề bài (ví dụ: phân số, phần trăm, nhân tử, ...) cũng có thể để 'result_unit' là rỗng. Với bài làm của học sinh, 'result_unit' có thể thiếu do học sinh quên viết đơn vị, với trường hợp này hãy ghi là missing.
- 'result_grand_unit' là đơn vị của thực thể được đối chiếu theo target (ví dụ 'unit' của sách là books, 'unit' của vở là pens nhưng 'unit' là tổng sách và bút (target) lại là items thì 'grand_unit' của sách và bút sẽ là items nhưng nếu target là tổng số bút ('unit' là pens) thì 'grand_unit' của sách vẫn là 'books' vì không liên quan nên sách không liên quan). Các thực thể mang tính scalar trong đề bài (ví dụ: phân số, phần trăm, nhân tử, ...) cũng có thể để 'result_grand_unit' là rỗng.
- 'reported_expr' là biểu thức tính toán trong lời giải học sinh (biểu thức có cả phép tính lẫn dấu '=', giá trị sau dấu '=' là giá trị của 'result' được học sinh tính toán ra).

Khi hoàn thành 'Output/StudentPlan.yaml', nó sẽ thêm vào cuối file dòng target: thực mà học sinh chọn ở phần kết luận phòng khi học sinh tính được target trong đề bài nhưng lại dùng thực thể khác làm target.

Nếu bài làm của học sinh có lỗi sai chính tả, LLM sẽ trả thêm lỗi "spelling errors" vào file 'Output/Diagnosis.yaml'. Ngoài ra nếu bài làm của học sinh trả lời tối nghĩa, không có dẫn giải thì LLM sẽ trả lỗi "word problem", nếu bài làm của học sinh trả lời bằng chữ thì sẽ thêm nhãn "answer by word" vào file 'Output/Diagnosis.yaml'. Các nhãn đều này có trường 'step' và 'entity' là rỗng (gán trường này do code python làm). Nếu có các lỗi này thì hãy ghi "No" vào file 'Output/Wrong.yaml'

▼ One-shot

▼ Input

Ví dụ bài toán trong file 'Input/Problem.txt':

"Nancy buys 2 coffees a day. She grabs a double espresso for \$3.00 every morning. In the afternoon, she grabs an iced coffee for \$2.50. After 20 days, how much money has she spent on coffee?"

Đây là danh sách thực thể trong 'Output/ProblemEntities.yaml':

morning_coffee_price:

'value': 3.00

'unit': dollars

'location': input

'grand_unit': dollars

afternoon_coffee_price:

'value': 2.50

'unit': dollars

'location': input

'grand_unit': dollars

days:

'value': 20

'unit': days

'location': input

'grand_unit':

total_cost:

'value':

'unit': dollars

'location': target

'grand_unit': dollars

Ví dụ với bài giải học sinh trong file 'Input/StudentAnswer.txt':

"Each morning coffee costs \$3.00.

Each afternoon coffee costs \$2.00.

Total cost per day is $\$3.00 + \$2.00 = \$5.00$.

She buys coffee for 20 days.

Total cost for 20 days is

$\$5.00 + 20 = \25.00 .

Therefore, the final answer is 25.0."

▼ Output

Tôi muốn output trong file 'Output/StudentPlan.yaml' là danh sách các bước như sau:

step1:

'expr': morning_coffee_price + afternoon_coffee_price

'result': daily_cost

'result_unit': dollars

```
'grand_result_unit': dollars
'reported_expr': 3.00 + 2.00 = 5.00
```

step2:

```
'expr': daily_cost + days
'result': total_cost
'result_unit': dollars
'grand_result_unit': dollars
'reported_expr': 5.00 + 20 = 25.00.
```

target: total_cost

Khi tạo xong 'Output/StudentPlan.yaml', hãy thêm các thực thể 'result' vào file 'Output/StudentAnswerEntities.yaml' (thêm tất cả trường 'value' theo giá trị sau dấu bằng của 'reported_expr' của thực thể, 'unit' theo 'result_unit', 'location' là step thực thể đó được tính toán (chỉ thêm vào các thực thể chưa có 'location'), 'grand_unit' theo 'result_grand_unit'). Ngoài ra nó còn thêm trường 'expr' cho các thực thể trong file 'Output/StudentAnswerEntities.yaml'. Nếu thực thể có 'location' là input nó sẽ để trống 'expr', với các thực thể là 'result' trong các bước tính của file 'Output/StudentPlan.yaml' 'expr' là trường 'expr' trong file 'Output/StudentPlan.yaml'. Sau đó nó sẽ thêm trường 'formalized_expr' là trường chuẩn hoá 'expr' về phép tính của các thực thể chỉ có 'location' là input, nếu thực thể có 'location' là input thì nó sẽ để trống 'formalized_expr'. Tất cả việc thêm đều tạo bằng code python. Nếu sinh plan thành công, chương trình sẽ ghi "Pass StudentAnswerFormalizer", ngược lại nếu thất bại sẽ ghi "Fail StudentAnswerFormalizer" vào 'Output/Log.yaml'.

▼ One-shot

▼ Input

Ví dụ danh sách thực thể trong file 'Output/StudentAnswerEntities.yaml':

```
morning_coffee_price:
  'value': 3.00
  'unit': dollars
  'location': input
  'grand_unit': dollars

afternoon_coffee_price:
  'value': 2.50
  'unit': dollars
```


'location': input
'grand_unit': dollars

days:

'value': 20
'unit': days
'location': input
'grand_unit':

total_cost:

'value':
'unit': dollars
'location': target
'grand_unit': dollars

daily_cost:

'value':
'unit': dollars
'location': step1
'grand_unit': dollars

Kế hoạch trong file 'Output/StudentPlan.yaml':

step1:

'expr': morning_coffee_price + afternoon_coffee_price
'result': daily_cost
'result_unit': dollars
'grand_result_unit': dollars
'reported_expr': $3.00 + 2.00 = 5.00$

step2:

'expr': daily_cost + days
'result': total_cost
'result_unit': dollars
'grand_result_unit': dollars
'reported_expr': $5.00 + 20 = 25.00$.

target: total_cost

▼ Output

Tôi muốn chương trình sẽ update file
'Output/StudentAnswerEntities.yaml' thành:
morning_coffee_price:

```

    'value': 3.00
    'unit': dollars
    'location': input
    'grand_unit': dollars
    'expr':
    'formalized_expr':

afternoon_coffee_price:
    'value': 2.50
    'unit': dollars
    'location': input
    'grand_unit': dollars
    'expr':
    'formalized_expr':

days:
    'value': 20
    'unit': days
    'location': input
    'grand_unit':
    'expr':
    'formalized_expr':

total_cost:
    'value': 25.00
    'unit': dollars
    'location': target
    'grand_unit': dollars
    'expr': daily_cost * days
    'formalized_expr': (morning_coffee_price + afternoon_coffee_price)
* days

daily_cost:
    'value': 5.00
    'unit': dollars
    'location': step1
    'grand_unit': dollars
    'expr': morning_coffee_price + afternoon_coffee_price
    'formalized_expr': morning_coffee_price + afternoon_coffee_price

```

▼ Code

Mapper

▼ Prompt

Hãy viết giúp tôi file 'Formalizer/Mapper.py' để map các thực thể trong file 'Output/StudentAnswerEntities.yaml' vào các thực thể trong file 'Output/PlanEntities.yaml'. Do 2 file này đã có các thực thể chung từ 'Output/ProblemEntities.yaml' nên các thực thể từ đầu đến thực thể có 'location' là target sẽ được auto map với nhau. Việc map các thực thể sau phần thực thể có 'location' là target sẽ được map theo quan hệ của các thực thể trong trường 'expr' (việc map dựa trên các thực thể đã map, không cần theo từng bước vì có thể bước trước biểu diễn quan hệ sai nhưng bước sau lại biểu diễn quan hệ đúng nên map xảy ra ở 2 đầu (từ step1 và từ 'expr' của thực thể có 'location' là target để map). Nó sẽ thêm trường 'map' vào toàn bộ thực thể trong danh sách thực thể trong file 'Output/StudentAnswerEntities.yaml' và 'Output/ProblemEntities.yaml' (trường này có thể để trống nếu có thực thể không map được). Toàn bộ file này chạy bằng code python (không dùng LLM).

▼ One-shot

▼ Input

Ví dụ danh sách thực thể trong file 'Output/PlanEntities.yaml':

morning_coffee_price:

'value': 3.00
'unit': dollars
'location': input
'grand_unit': dollars
'expr':
'formalized_expr':

afternoon_coffee_price:

'value': 2.50
'unit': dollars
'location': input
'grand_unit': dollars
'expr':
'formalized_expr':

days:

'value': 20

```

    'unit': days
    'location': input
    'grand_unit':
    'expr':
    'formalized_expr':
total_cost:
    'value': 110.00
    'unit': dollars
    'location': target
    'grand_unit': dollars
    'expr': daily_cost * days
    'formalized_expr': (morning_coffee_price + afternoon_coffee_price)
* days
daily_cost:
    'value': 5.50
    'unit': dollars
    'location': step1
    'grand_unit': dollars
    'expr': morning_coffee_price + afternoon_coffee_price
    'formalized_expr': morning_coffee_price + afternoon_coffee_price

```

'Output/StudentAnswerEntities.yaml' là danh sách các thực thể như sau:

```

morning_coffee_price:
    'value': 3.00
    'unit': dollars
    'location': input
    'grand_unit': dollars
    'expr':
    'formalized_expr':
afternoon_coffee_price:
    'value': 2.50
    'unit': dollars
    'location': input
    'grand_unit': dollars
    'expr':
    'formalized_expr':

```

```

days:
  'value': 20
  'unit': days
  'location': input
  'grand_unit':
  'expr':
  'formalized_expr':

total_cost:
  'value': 25.00
  'unit': dollars
  'location': target
  'grand_unit': dollars
  'expr': daily_cost * days
  'formalized_expr': (morning_coffee_price + afternoon_coffee_price)
* days

daily_cost:
  'value': 5.00
  'unit': dollars
  'location': step1
  'grand_unit': dollars
  'expr': morning_coffee_price + afternoon_coffee_price
  'formalized_expr': morning_coffee_price + afternoon_coffee_price

```

▼ Output

Tôi muốn chương trình sẽ update danh sách thực thể trong file
'Output/PlanEntities.yaml' thành:

```

morning_coffee_price:
  'value': 3.00
  'unit': dollars
  'location': input
  'grand_unit': dollars
  'expr':
  'formalized_expr':
  'map': morning_coffee_price

afternoon_coffee_price:
  'value': 2.50
  'unit': dollars
  'location': input

```

```

    'grand_unit': dollars
    'expr':
    'formalized_expr':
    'map': afternoon_coffee_price

days:
    'value': 20
    'unit': days
    'location': input
    'grand_unit':
    'expr':
    'formalized_expr':
    'map': days

total_cost:
    'value': 110.00
    'unit': dollars
    'location': target
    'grand_unit': dollars
    'expr': daily_cost * days
    'formalized_expr': (morning_coffee_price + afternoon_coffee_price)
* days
    'map': total_cost

daily_cost:
    'value': 5.50
    'unit': dollars
    'location': step1
    'grand_unit': dollars
    'expr': morning_coffee_price + afternoon_coffee_price
    'formalized_expr': morning_coffee_price + afternoon_coffee_price
    'map': daily_cost

```

File 'Output/StudentAnswerEntities.yaml' cũng sẽ được update như sau:

```

morning_coffee_price:
    'value': 3.00
    'unit': dollars
    'location': input
    'grand_unit': dollars

```

```

    'expr':
    'formalized_expr':
    'map': morning_coffe_price
afternoon_coffee_price:
    'value': 2.50
    'unit': dollars
    'location': input
    'grand_unit': dollars
    'expr':
    'formalized_expr':
    'map': afternoon_coffee_price
days:
    'value': 20
    'unit': days
    'location': input
    'grand_unit':
    'expr':
    'formalized_expr':
    'map': days
total_cost:
    'value': 25.00
    'unit': dollars
    'location': target
    'grand_unit': dollars
    'expr': daily_cost * days
    'formalized_expr': (morning_coffee_price + afternoon_coffee_price)
* days
    'map': total_cost
daily_cost:
    'value': 5.00
    'unit': dollars
    'location': step1
    'grand_unit': dollars
    'expr': morning_coffee_price + afternoon_coffee_price
    'formalized_expr': morning_coffee_price + afternoon_coffee_price
    'map': daily_cost

```

▼ Code

InsideChecker

▼ Prompt

Hãy viết giúp tôi file 'Verify/InsideChecker.py' để kiểm tra các lỗi sai nội tại của 1 bài giải qua plan và entities. Nó có 2 chế độ là check lời giải LLM và lời giải học sinh. Nếu chế độ check lời giải LLM chương trình sẽ nhận file 'Output/Plan.yaml' và 'Output/PlanEntities' và xuất output vào file 'Output/Error.yaml'. Nếu là chế độ check lời giải học sinh, chương trình sẽ nhận file 'Output/StudentPlan.yaml' và 'Output/StudentAnswerEntities.yaml' và xuất output vào file 'Output/Diagnosis.yaml'. Nó gồm các hàm check sau:

- Hàm kiểm tra sai mục tiêu: Hàm này sẽ kiểm tra thực thể nào có 'location' là target (trong file 'Output/PlanEntities' hoặc 'Output/StudentAnswerEntities.yaml') đã được tính toán chưa. Ngoài ra nó còn kiểm tra thực thể ở cuối file 'Output/StudentPlan.yaml' (target : entity) có phải thực thể 'location' là target hay không. Nếu có lỗi, nó sẽ trả vào output ('Output/Error.yaml' hoặc 'Output/Diagnosis.yaml').
 - "wrong target"
 - step:
 - entity:
- Hàm kiểm tra tính toán sai: Hàm này sẽ kiểm tra phép tính (kiểm tra tính toán bằng máy tính) ở trường 'reported_expr' (trong 'Output/Plan.yaml' hoặc 'Output/StudentPlan.yaml'). Nếu có lỗi nó sẽ trả vào output:
 - "wrong calculation"
 - step: step i (với step i là step có 'reported_expr' bị sai vào output).
 - entity: là thực thể 'result' của step i
- Hàm kiểm tra thiếu đơn vị: Hàm này chỉ hoạt động khi input là lời giải học sinh. Nó sẽ kiểm tra các thực thể trong 'Output/StudentAnswerEntities.yaml', thực thể nào có 'unit' là missing thì là thiếu đơn vị. Nếu thiếu nó sẽ ghi vào outputL
 - "unit missing"
 - step:

- entity: entity x (với entity x là thực thể có 'unit' là missing).
- Hàm kiểm tra chỉ trả lời đáp án: Hàm này sẽ kiểm tra các kế hoạch trong input ('Output/Plan.yaml' hoặc 'Ouput/StudentPlan.yaml'). Nếu kế hoạch chỉ có 1 bước và 'expr' để rỗng thì hàm này sẽ trả lỗi. Nó sẽ trả vào output:
 - "only final answer"
 - step:
 - entity:
- Hàm kiểm tra sai quan hệ: Hàm này sẽ trong các bước trong kế hoạch trong input ('Output/Plan.yaml' hoặc 'Ouput/StudentPlan.yaml') nếu 1 bước có 'expr' có các thực thể có 'unit' khác nhau và 'grand_unit' khác nhau mà cộng hoặc trừ với nhau thì nó sẽ trả vào output:
 - "wrong relationship" nếu 'unit', 'grand_unit' khác các đơn vị cần đổi như độ dài, thể tích, diện tích, tá, ...
 - step: step i (với step i là step có 'expr' bị sai)
 - entity:
 - "do not convert units" nếu 'unit', 'grand_unit' là các đơn vị cần đổi như độ dài, thể tích, diện tích, tá, ...
 - step: step i (với step i là step có 'expr' bị sai)
 - entity:
- Hàm kiểm tra thiếu bước: Hàm kiểm tra xem khi thực hiện theo thứ tự kế hoạch trong input ('Output/PlanEntities.yaml' hoặc 'Ouput/StudentAnswerEntities.yaml'). Với thực thể có 'location' là target, nó sẽ kiểm tra trường 'expr' xem các thực thể trong biểu thức đã tồn tại chưa và có 'value' khác trống không. Với thực thể có 'location' khác target và input, nếu trong thực thể có 'expr' chứa thực thể đứng sau nó hoặc chưa được tính toán hoặc không tồn tại trong file ('Output/PlanEntities.yaml' hoặc 'Ouput/StudentAnswerEntities.yaml') thì nó sẽ trả lỗi về output
 - "missing step"
 - step: step i (step i là bước có thực thể chưa tồn tại).

- entity: entity x (entity x là thực thể được phát hiện chưa tồn tại hoặc tính toán)
- Hàm kiểm tra đọc sai đề và lỗi logic: Hàm kiểm tra xem khi thực hiện theo thứ tự kế hoạch trong input ('Output/Plan.yaml' hoặc 'Output/StudentPlan.yaml'), nếu trong 1 bước có các giá trị trong 'reported_expr' khác giá trị 'value' của các thực thể trong 'expr'. Nếu thực thể bị khác giá trị có 'location' là input thì trả lỗi vào output
 - "misreading"
 - step: step i (step i là bước bị sai).
 - entity: entity x (entity x là thực thể có 'value' khác giá trị trong 'reported_expr')

Nếu 'location' khác input thì trả lỗi

- "logic error"
 - step: step i (step i là bước bị sai).
 - entity: entity x (entity x là thực thể có 'value' khác giá trị trong 'reported_expr')
- Hàm kiểm tra thừa bước: Hàm này sẽ kiểm tra trong các bước thực hiện kế hoạch có 1 thực thể được tạo ra nhưng sau đó không được dùng đến trong các bước sau không (ngoại trừ thực thể có 'location' là target). Nếu có thì nó sẽ trả vào output
 - "extra step"
 - step: step i (step i là bước có 'result' là thực thể thừa)
 - entity: entity x (entity x là thực thể bị thừa)

Nó sẽ bỏ bước thừa đi (tạo file copy kế hoạch trong input và bỏ trong file copy) và quét đến khi không còn bước thừa nữa để in lỗi chính xác hơn.

Trả lỗi "extra step", nếu các hàm trả lỗi khác hãy ghi "Yes" vào 'Output/Wrong.yaml'. Nếu lỗi "extra step" thì ghi vào là "No" vào 'Output/Wrong.yaml'

▼ Code

CompareChecker

▼ Prompt

Hãy viết giúp tôi file 'Verify/CompareChecker.py' để kiểm tra các lỗi sai khi so sánh danh sách thực thể và kế hoặc 2 bài giải với nhau. Chương trình sẽ nhận file 'Output/Plan.yaml' và 'Output/PlanEntities' và 'Output/StudentPlan.yaml' và 'Output/StudentAnswerEntities.yaml' và xuất output vào file 'Output/Diagnosis.yaml'. Nó gồm các hàm check sau:

- Hàm kiểm tra đổi đơn vị sai: Nó sẽ kiểm tra các thực thể trong file 'Output/StudentAnswerEntities.yaml' đã được map (có 'map' khác rỗng) với các thực thể trong file 'Output/PlanEntities' mà có 'value' khác nhau. Nếu nó có 'unit', 'grand_unit' là các đơn vị cần đổi như độ dài, thể tích, diện tích, tá, thì sẽ trả về lỗi:
 - "wrong units conversion"
 - step:
 - entity: entity x (entity x là thực thể sai)
- Hàm kiểm tra tách bước, đảo bước, gộp bước: Nó sẽ so sánh hai trường 'formalized_expr' của thực thể có 'location' là target trong 2 file 'Output/StudentAnswerEntities.yaml' và 'Output/PlanEntities.yaml'. Nếu 2 trường này có phép tính tương đương nhau (có thể kiểm tra bằng check danh sách thực thể và phép tính hoặc thay 3 lần bộ số ngẫu nhiên cho các thực thể trong 'formalized_expr', nếu chúng ra kết quả bằng nhau thì là tương đương). Nó sẽ trả về lỗi:
 - "combine step" nếu số lượng bước trong lời giải học sinh ít hơn bước trong lời giải chuẩn
 - step:
 - entity:
 - "step separation" nếu số lượng bước trong lời giải học sinh nhiều hơn bước trong lời giải chuẩn
 - step:
 - entity:
 - "reverse steps" nếu số lượng bước trong lời giải học sinh bằng số bước trong lời giải chuẩn

- step:
- entity:
- Hàm kiểm tra hoàn toàn khớp: Nếu toàn bộ các thực thể trong 2 lời giải có trường 'map', 'value', 'unit', 'expr' khớp nhau thì nó sẽ trả vào output:
 - "all right"
 - step:
 - entity:
- Hàm kiểm tra sai quan hệ: Hàm này sẽ kiểm tra nếu 1 thực thể trong file 'Output/StudentAnswerEntities.yaml' đã được có 'map' được là 1 thực thể trong file 'Output/PlanEntities.yaml' và có 'expr' khác thì trả về lỗi vào output:
 - "wrong relationship"
 - step: step i (step i là step trong bài làm của học sinh 'Output/StudentPlan.yaml' có 'expr' giống với 'expr' của thực thể đang xét)
 - entity:
- Hàm kiểm tra cách khác: hàm này sẽ kiểm tra nếu thực thể có 'location' là target trong file 'Output/StudentAnswerEntities.yaml' khác 'formalized_expr' so với Output/PlanEntities.yaml' nhưng đáp án lại đúng. Nó sẽ thay 3 lần biến ngẫu nhiên vào làm giá trị các thực thể trong 'formalized_expr', nếu cả 3 lần 'value' thực thể có 'location' ở 2 file đều trùng nhau thì nó sẽ trả lỗi vào output:
 - "different calculation"
 - step:
 - entity:

Trừ lỗi "wrong relationship", nếu các hàm trả lỗi khác hãy ghi "No" vào 'Output/Wrong.yaml'. Nếu lỗi "wrong relationship" thì ghi vào là "Yes" vào 'Output/Wrong.yaml'

▼ Code

LLMChecker

▼ Prompt

Hãy viết giúp tôi file 'Verifier/LLMChecker.py'

▼ Code